

```
moflix.py      x
14         if genre=="0" :
15             return ("In a python array form, give me " + str(number_recs) +" movie recommendation, similar to " +
16             if similar=="0":
17                 return ("In a python array form, give me "+genre + " "+ str(number_recs) +" movie recommendations")
18             return ("In a python array form, give me "+genre +" "+str(number_recs)+" movie recommendation similar to
19
20 def Res(x):
21     response = openai.Completion.create(
22         model="text-davinci-002",
23         prompt=x,
24         top_p=1,
25         max_tokens=100)
26     return(response['choices'][0]['text']).strip())
27
28 def InBox(x):
29     x=(["Movie Recommendations"]+x)
30     #this gives you the longest string
31     biggest = max(x, key = len)
32     #THIS GETS the len of the biggest string
33     biggest_len=len(biggest)
```

```
65% (39:1)
Terminal      x
codio@slowgroup-sugarcosmos:~/workspace$ python3 moflix.py 9
Enter 0, if you don't have an answer in mind
How many movies do you want recommended: 0
What genre are you looking for: 0
What is a similar movie: star wars
+-----+
| Movie Recommendations |
+-----+
| A New Hope             |
+-----+
| The Empire Strikes Back|
+-----+
| Return of the Jedi     |
+-----+
| The Phantom Menace     |
+-----+
| Attack of the Clones   |
+-----+
codio@slowgroup-sugarcosmos:~/workspace$
```

Part 3. Presenting Info

Now that we have our prompt let's take care of integrating our OpenAI API key as we have done before . Ideally you will move to the top of our file.

```
import os
import openai
import secret
openai.api_key=secret.api_key
```

Let's create a quick function that takes a prompt and returns our movie results . This function should go between our `InBox` and `Moreflix` functions. We are going to call this function `Res` short for response. This function should work as all the previous prompts we generated using our API.

```
def Res(x):
    response = openai.Completion.create(
        model="text-davinci-002",
        prompt=x,
        top_p=1,
        max_tokens=100)
    return(response['choices'][0]['text']).strip())
```

TRY IT!

We can add the following at the end of our code to check how `Res` behaves.

```
Prompts=Moreflix() + "rec ="
print(Res(Prompts))
```

TRY IT!

After checking that everything works as intended we can change our print to a value assignment. Set the `Res` function call to the variable `movies` .
`movies=Res(Prompts)` .

We are at the finish line. We know that the response generated by GPT-3 was a string. Even though, it was a string we want to convert it to an actual list. After that we can call `InBox` from our list.

```
# Converting string to list
import ast
movie_list=ast.literal_eval(movies)
InBox(movie_list)
```

TRY IT!

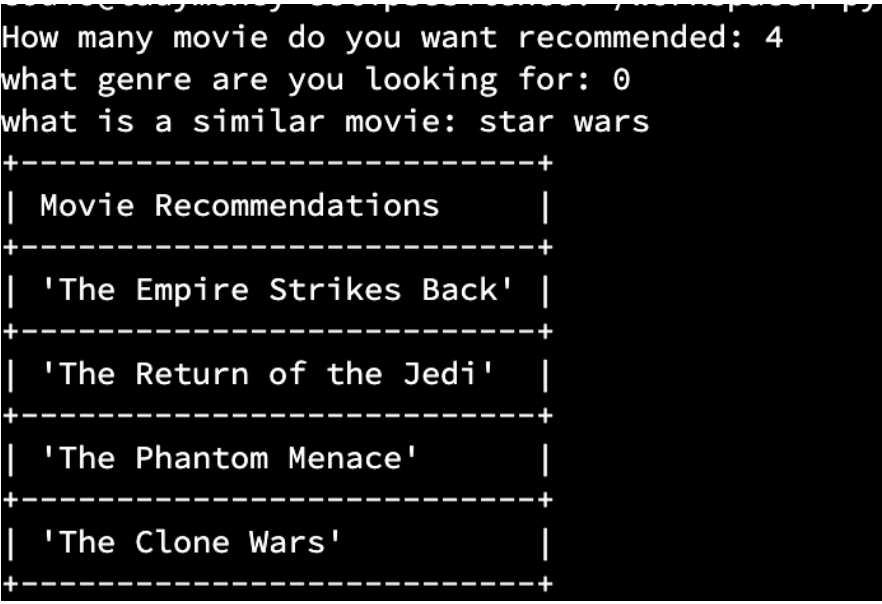
One last piece we are going to take care of are edge cases where the completions contains a numerated list or additional characters:

```
import ast

Prompts=Moreflix() + "rec ="
movies=(Res(Prompts))
try:
    movie_list=ast.literal_eval(movies)
except:
    movie_list2=""
    for i in movies:
        if i=="]":
            movie_list2+=(i)
            break
        else:
            movie_list2+=(i)
    movie_list=ast.literal_eval(movie_list2)
InBox(movie_list)
```

TRY IT!

Your program should be able to generate something like the image below.



If some reason your code is failing and you cannot understand why, feel free to compare your code with the following:

► [Click here to see the code solution](#)

What is the purpose of the `Res` function in the provided Python script?

☐

It is used to print a list of movie recommendations in a formatted box.

☐

It is used to collect user input about the number, genre, and similarity of desired movie recommendations.

☒

It is used to generate movie recommendations based on a specific prompt by invoking OpenAI's GPT model.☐

The `Res` function in the script takes an input prompt and sends it to OpenAI's GPT model. It uses the `openai.Completion.create` method to generate a text completion based on the input prompt. The function then returns this generated text. In the context of this script, the `Res` function is used to generate movie recommendations based on the input collected from the user (i.e., the number of recommendations, the genre, and a similar movie).

Check It!